Administración de Sistemas Operativos

David Ubide Alaiz



Gestión de Procesos

En cada momento se están ejecutando un gran número de procesos:

- Procesos de sistema (kernel, daemons)
- Procesos de usuarios

En esta sección trataremos la gestión de los procesos que se están ejecutando:

- Listar procesos en ejecución
- Detener y matar procesos
- Controlar la prioridad de ejecución

¿Qué es un proceso?

El concepto de proceso es precisamente eso, un flujo de ejecución del programa, identificado por un identificador de proceso único en el sistema (de nuevo, no tendremos que preocuparnos de rebasar el número máximo de identificadores, ya que es muy elevado)

¿Qué define un proceso?

- El código que ejecuta el proceso.
- Las variables globales del proceso.
- La pila asignada al proceso, donde se almacenan las variables locales.
- Los descriptores de ficheros.
- El estado de las señales.
- El usuario y grupo propietarios del proceso.
- El identificador del proceso padre.
- Información para contabilidad: tiempos, recursos consumidos, etc.

¿Qué define un proceso?



¿Qué define un proceso?

- Un proceso tiene su propio espacio de memoria, que incluye el código, los datos y los recursos asignados.
- Los procesos se ejecutan de forma independiente y están aislados de otros procesos.
- Los procesos no pueden acceder directamente a la memoria o recursos de otros procesos.
- Los procesos pueden crearse, suspenderse, reanudarse y terminarse de forma independiente.
- Los procesos suelen tener un hilo principal que inicia la ejecución.
- Los procesos pueden crear hilos secundarios adicionales dentro del mismo proceso.

https://medium.com/@diego.coder/entendiendo-los-procesos-hilos-y-multihilos-9423f6e40ca7

¿Qué es un hilo?

Un hilo, o también llamado subproceso, es básicamente un flujo de ejecución o una secuencia de instrucciones manejada de forma independiente. Un hilo vive dentro del contexto de un proceso y este utiliza todos sus recursos; como por ejemplo, la memoria o los registros

https://medium.com/@diego.coder/entendiendo-los-procesos-hilos-y-multihilos-9423f6e40ca7

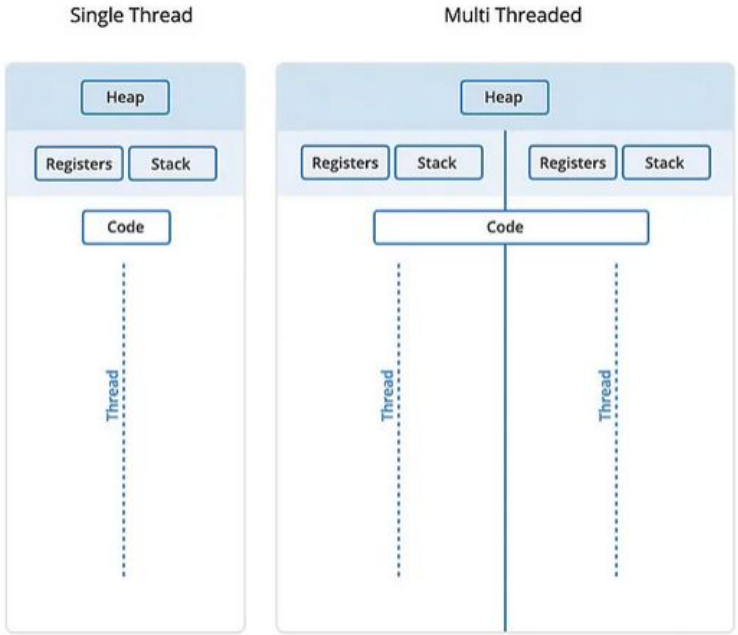
¿Qué es el multihilo?

Multihilo o también conocido como multithreading, se refiere a la capacidad de un sistema o programa de ejecutar múltiples hilos de forma concurrente dentro de un proceso.

En un entorno de ejecución de un solo hilo, el programa se ejecuta secuencialmente, es decir, una instrucción se ejecuta después de la otra y en orden. En cambio, en un entorno de ejecución multihilo, el programa puede dividirse en múltiples hilos, cada uno de los cuales se ejecuta simultáneamente y realiza un conjunto específico de tareas.

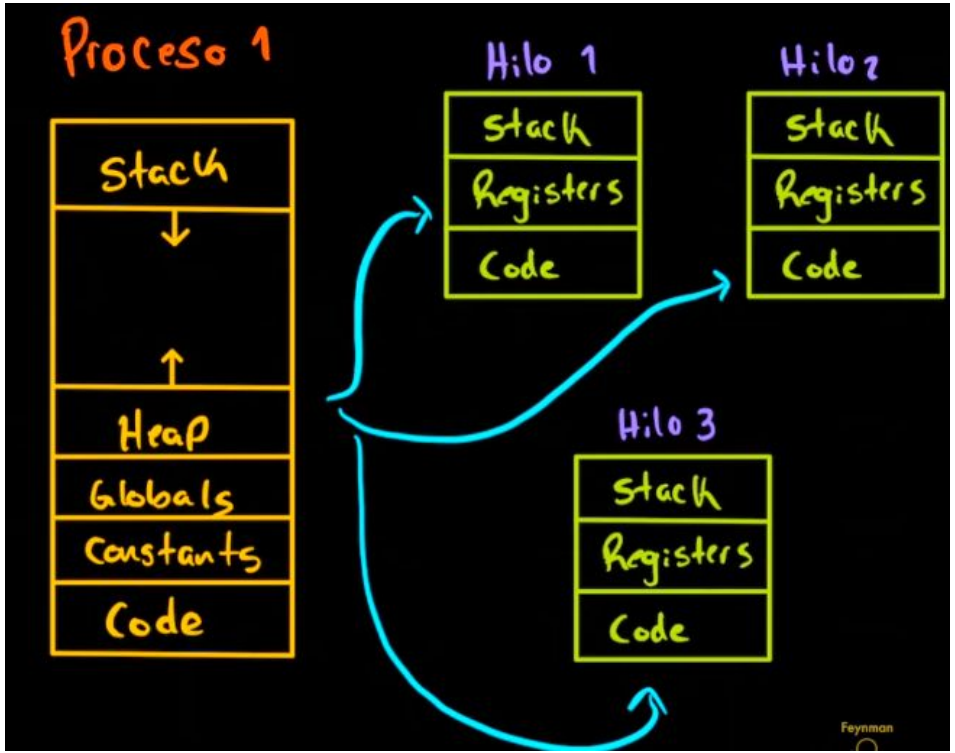
https://medium.com/@diego.coder/entendiendo-los-procesos-hilos-y-multihilos-9423f6e40ca7

¿Qué es el multihilo?



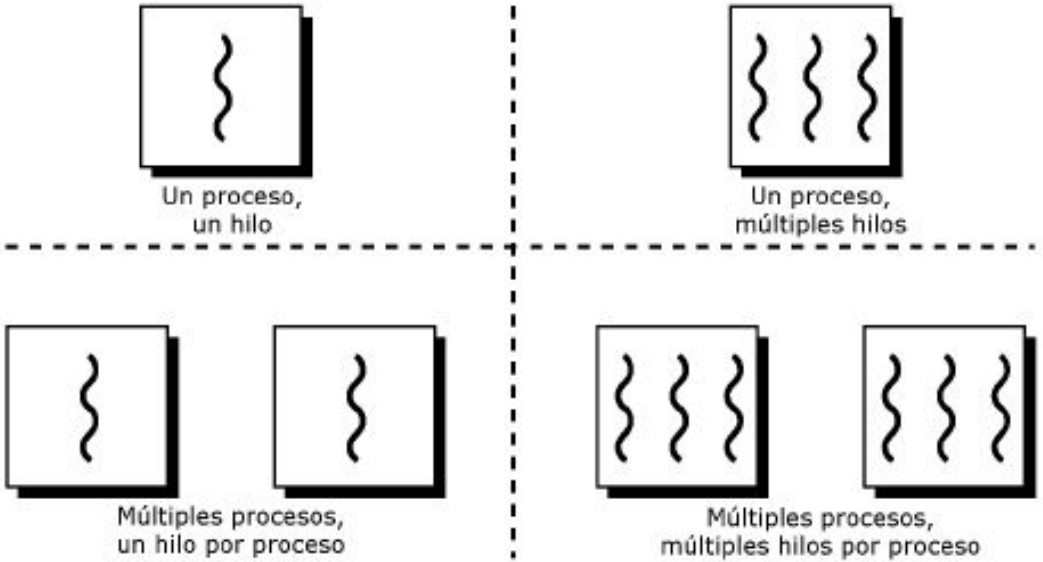
https://medium.com/@diego.coder/entendiendo-los-procesos-hilos-y-multihilos-9423f6e40ca7

¿Qué es el multihilo?



https://www.youtube.com/watch?v=n6IxKTS2zYs

¿Qué es el multihilo?



¿Qué es un trabajo?

Un trabajo (job) es una unidad de trabajo gestionada por el shell o el planificador del sistema.
Puede contener uno o varios procesos, que el usuario ejecuta en primer o segundo plano.

Características principales:

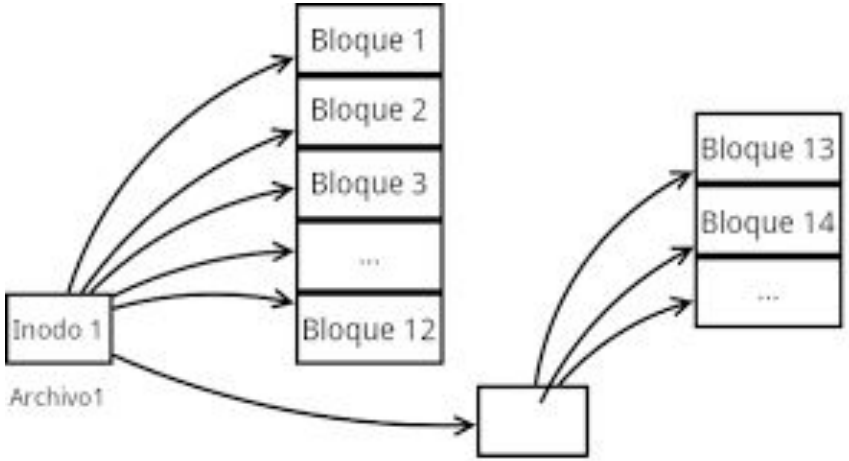
- Se identifica con un Job ID (%1, %2, etc.).
- Puedes suspender, reanudar o enviar al fondo los trabajos.
- Suelen estar planificados

¿Qué es un trabajo?

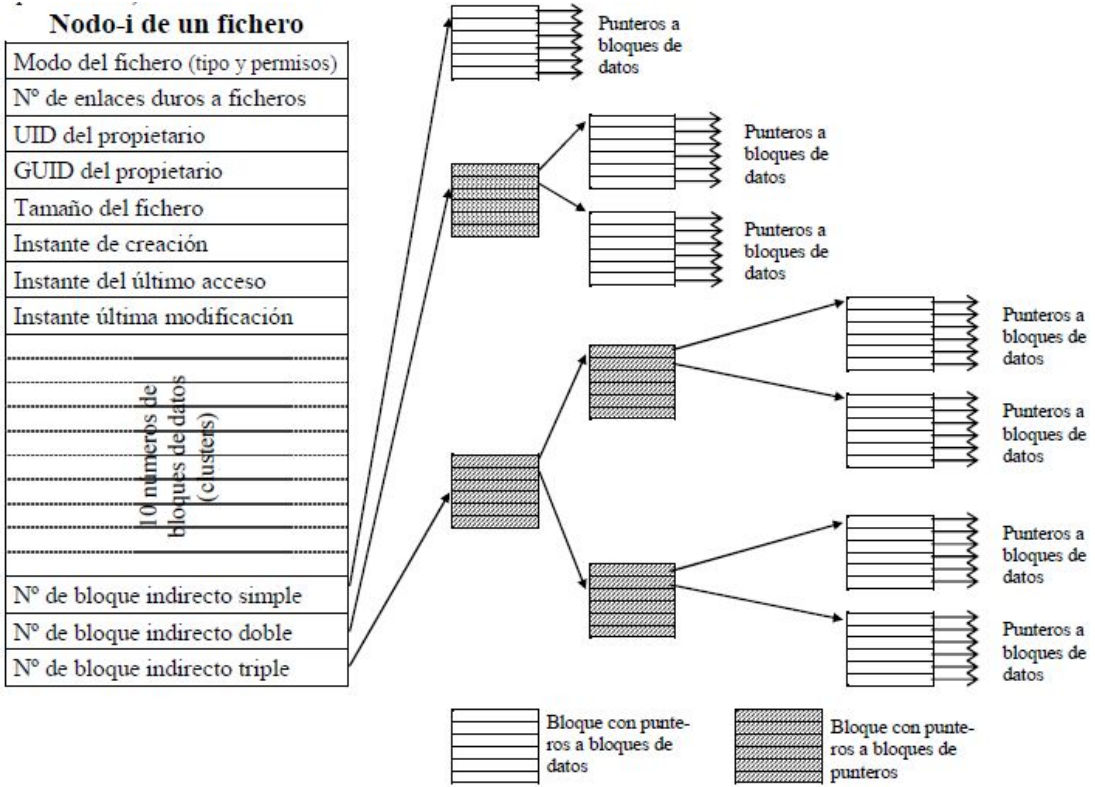
Concepto	Qué representa	Identificador	Espacio de memoria	Comunicación	Ejemplo
Proceso	Programa en ejecución	PID	Propio e independiente	IPC (pipes, colas…)	`firefox`, `bash`, `sshd`
Hilo	Subproceso dentro de un proceso	TID (Thread ID)	Compartido con otros hilos del mismo proceso	Variables compartidas	Hilos de descargas en Firefox
Trabajo	Grupo de procesos gestionado por el shell	Job ID (`%1`, `%2`)	Depende de los procesos que lo forman	No aplica directamente	`sleep 100 &`, `fg`, `bg`

¿Que son los i-nodos?

El término "inodo" refiere generalmente a inodos en discos (dispositivos en modo bloque) que almacenan archivos regulares, directorios, y enlaces simbólicos. El concepto es particularmente importante para la recuperación de los sistemas de archivos dañados.



¿Que son los i-nodos?



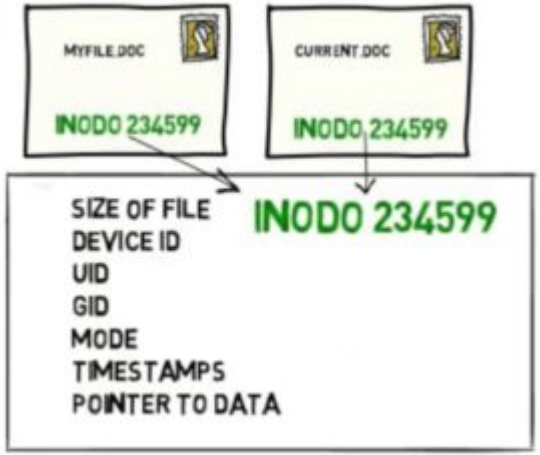
¿Que son los i-nodos?

En Linux los archivos no se referencian por su nombre. Se referencian por su número de inodo. Gracias a esto en Linux podemos usar enlaces duros “Hard link”. Ya que la tabla de nombres la maneja el kernel. A continuación vemos un ejemplo de 2 nombres de la tabla del kernel que apuntan al mismo nodo.

```
ln TARGET LINK_NAME
```

```
ln test.txt enlace-duro-a-test.txt
```

```
ls -li
786433 -rw-r--r-- 2 mortadelo mortadelo 0 jun 21 21:27
enlace-duro-a-test.txt
786433 -rw-r--r-- 2 mortadelo mortadelo 0 jun 21 21:27
test.txt
```



¿Que son los enlaces simbólicos?

Un symlink (también llamado enlace simbólico) es un tipo de archivo en Linux que apunta a otro archivo o una carpeta en tu computador. Los enlaces simbólicos son similares a los accesos directos en Windows.

ln -s <ruta del archivo/Carpeta a ser enlazado> <ruta del enlace que se creará>



https://www.fpgenred.es/GNU-Linux/enlace_simblico.html

Tipos de procesos

Child (hijos)

Son procesos creados por otro proceso durante su ejecución. Usualmente los procesos child son creados para ejecutar un binario desde un proceso existente, con la llamada del sistema fork(). Los procesos normalmente son creados para ejecutarse a través de un shell o terminal. En dicho caso el shell se convierte en proceso padre y el proceso ejecutado se convierte en hijo. En sistemas tipo Unix/Linux cada proceso tiene un padre excepto el proceso init.



Tipos de procesos

UNIX Daemon (demonios)

Son tipos especiales de procesos que se ejecutan en segundo plano y están relacionados con el sistema operativo y no tienen shell asociado. Estos corren con permisos de root y usualmente proveen servicios. El no tener shell asociado se logra separando el proceso del shell, creando un proceso nuevo y terminando el proceso padre (el shell que lo inició). Desde el momento en que su proceso padre es terminado el proceso de hace independiente (ya no es más un proceso child) y es tomado por el proceso init el cual lo convierte en un daemon.



Tipos de procesos

UNIX Daemon (demonios)

En systemd los demonios se definen y configuran en los llamados archivos de unidad, unit files. Estos archivos se alojan en /etc/systemd/system/

Crear un demonio que se encargue de iniciar y vigilar un hipotético script (volver a iniciarlo si se para, reiniciarlo cuando el sistema se reinicie…). Para ello usaremos lo que en el argot de systemd se llama service, unit files con extensión *.service*



Tipos de procesos

UnDemonio.service

[Unit]

Description=La descripción del demonio

After=multi-user.target

[Service]

Type=simple

ExecStart=/ruta/hasta/el/script

User=usuario

WorkingDirectory=/ruta/hasta/la/carpeta/del/script

Restart=on-failure

StandardOutput=syslog

StandardError=syslog

[Install]

WantedBy=multi-user.target

https://voragine.net/linux/como-crear-y-gestionar-demonios-en-linux-con-systemd



Tipos de procesos

```
sudo systemctl daemon-reload
sudo systemctl enable undemonio.service
sudo systemctl start undemonio.service
```

https://voragine.net/linux/como-crear-y-gestionar-demonios-en-linux-con-systemd



Tipos de procesos

```
***MusicBot.service***
[Unit]
Description=MusicBot
After=multi-user.target

[Service]
Type=simple
ExecStart=/home/pi/bot/MusicBot/run.py
User=pi
WorkingDirectory=/home/pi/bot/MusicBot
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

https://www.reddit.com/r/raspberry_pi/comments/4vhofs/creating_a_systemd_daemon_so_you_can_run_a_python/



Tipos de procesos

```
***MusicBot.service***
[Unit]
Description=MusicBot
After=multi-user.target

[Service]
Type=simple
ExecStart=/home/pi/bot/MusicBot/run.py
User=pi
WorkingDirectory=/home/pi/bot/MusicBot
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

https://www.reddit.com/r/raspberry_pi/comments/4vhofs/creating_a_systemd_daemon_so_you_can_run_a_python/



Corriendo un script en Linux





Tipos de procesos

Windows Servicios (daemon)

Un servicio (Windows Service) es un programa que opera en segundo plano, sin intervención del usuario, y que puede iniciarse automáticamente al arranque, manualmente o ante ciertos eventos.

Los servicios en Windows están gestionados por el Service Control Manager (SCM), que es la estructura del sistema encargada de arrancar, detener, pausar, reanudar, supervisar servicios.





Tipos de procesos

Windows Servicios (daemon)

Un servicio debe cumplir ciertas reglas: por ejemplo, responder a comandos de control (start, stop, pause, resume), funcionar sin necesidad de usuario logueado (o con credenciales del sistema), operar en segundo plano.

Hasta Windows Vista, algunos servicios “interactivos” podían interactuar con la interfaz gráfica. Pero desde Vista en adelante se ha limitado mucho esa capacidad por seguridad (“interactive services” están deshabilitados o fuertemente restringidos





Tipos de procesos

Ejemplos

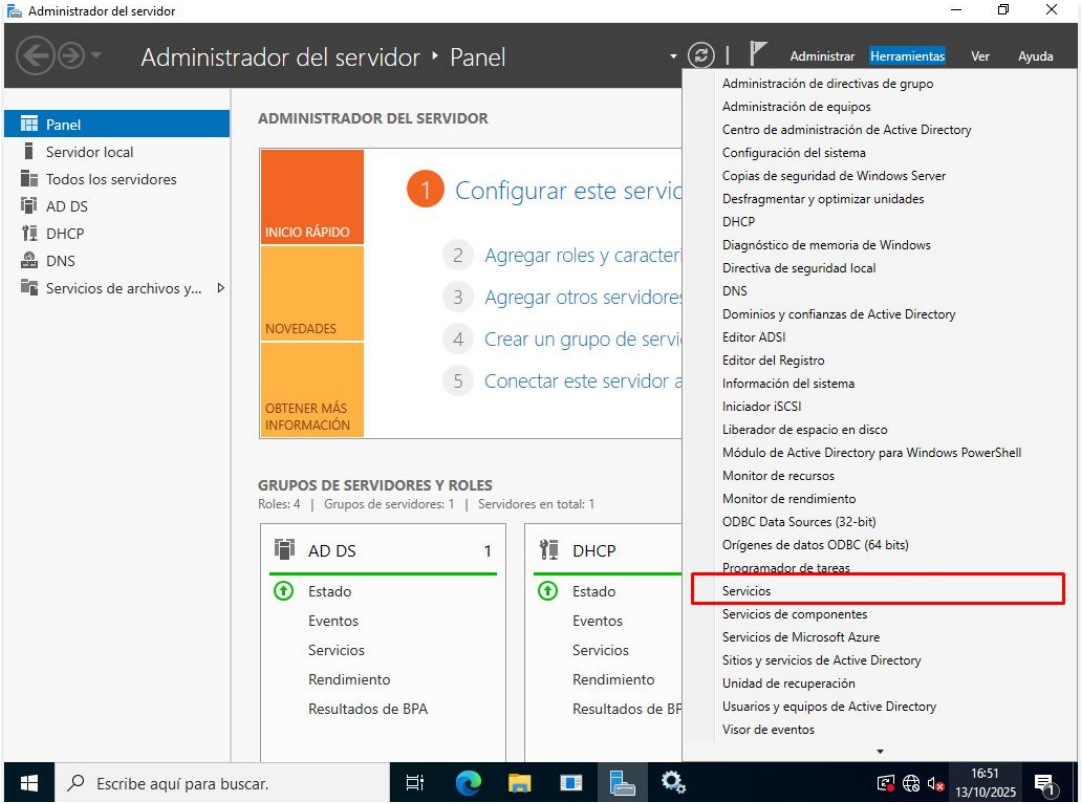
Un servicio debe cumplir ciertas reglas: por ejemplo, responder a comandos de control (start, stop, pause, resume), funcionar sin necesidad de usuario logueado (o con credenciales del sistema), operar en segundo plano.

Hasta Windows Vista, algunos servicios “interactivos” podían interactuar con la interfaz gráfica. Pero desde Vista en adelante se ha limitado mucho esa capacidad por seguridad (“interactive services” están deshabilitados o fuertemente restringidos

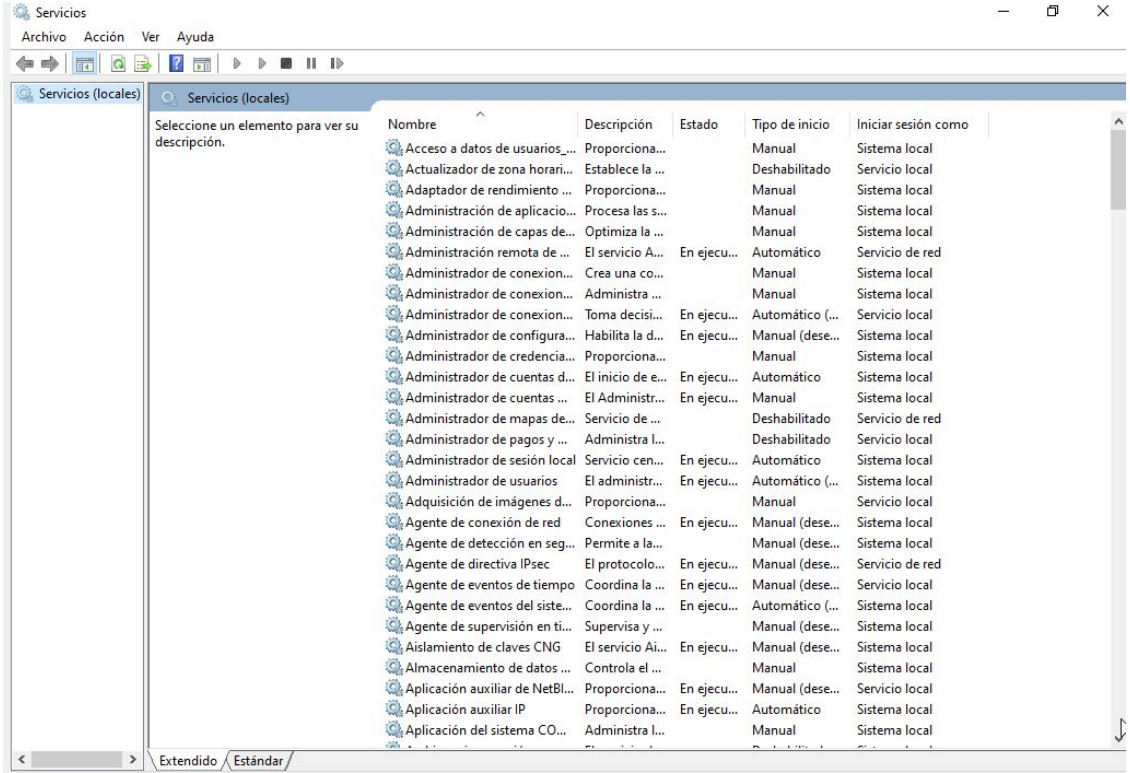




Tipos de procesos



Tipos de procesos



Ahora tú…

Lístame los 5 servicios de tipo demonio en Windows y dime dónde se encuentran



Tipos de procesos

Orphan (huérfanos)

Usualmente un proceso crea un proceso hijo (child) y cuando el proceso hijo termina una señal es emitida al proceso padre para que pueda hacer todo lo requerido cuando el proceso hijo es terminado. Pero hay situaciones en las que los procesos padres son matados (killed). En dicho caso el proceso hijo queda huérfano y entonces es tomado por el proceso init. Aún así el proceso cuyo padre fue matado sigue siendo llamado huérfano ya que su padre original no existe.



Tipos de procesos

Zombie

Cuando un proceso child o hijo es terminado o es completada su ejecución, luego su entrada en la tabla de procesos se mantiene hasta que el proceso padre obtenga la información del estado del proceso hijo terminado. Hasta entonces el proceso terminado entra en estado zombie y es conocido como proceso zombie. Cuando un proceso es terminado toda la memoria y recursos asociados con dicho proceso son liberados pero la entrada del mismo en la tabla de procesos aún existe. Una señal SIGCHILD es enviada al proceso padre (ha sido terminado).



Estado de los procesos

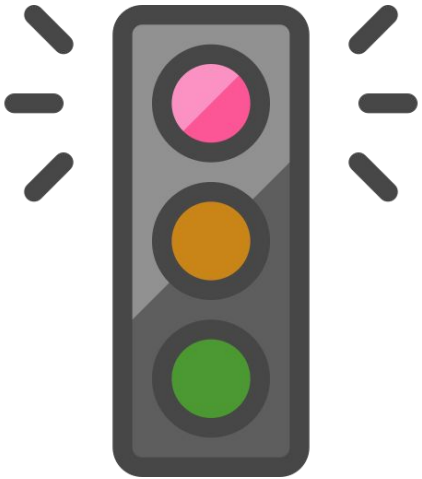
Activo: el proceso está empleando la CPU, por tanto, está ejecutándose. Puede haber tantos procesos activos como procesadores haya disponibles. Por tanto, si el sistema dispone de un único procesador, únicamente puede haber un proceso activo a la vez.

Preparado: el proceso no está ejecutándose, pero es candidato a pasar a estado activo. Es el planificador el que, en base a un criterio de planificación, decide qué proceso selecciona de la lista de procesos preparados para pasar a estado activo.

Bloqueado: el proceso está pendiente de un evento externo que le ha hecho bloquear, tales como una operación de lectura/escritura, la espera de finalización de un proceso hijo, una señal o una operación sobre un semáforo. El dispositivo/hecho externo "avisa" al S.O. cuando ha terminado la acción que realizaba mediante una INTERRUPCIÓN, dejando el S.O. lo que está haciendo para atender a esta última. Tras esto, el S.O. comprueba cuales son los procesos que fueron bloqueados por ese evento externo, cambiándolos al estado de preparado.

Estado de los procesos

En espera / Preparación: Estado por el que pasan los procesos antes de pasar a estar preparados por primera vez. Los procesos, cuando comienzan a existir, no están preparados para comenzar a ejecutar instrucciones hasta que el sistema no ha llevado a cabo una serie de actividades. Una vez que el proceso está completamente cargado, ya se puede producir la primera transición al estado preparado.



Estado de los procesos

Terminado: La transición de activo a este estado ocurre cuando el proceso realiza una llamada al sistema solicitando su propia terminación. En estas circunstancias, hay estructuras de datos correspondientes al proceso que no pueden ser liberadas hasta que el proceso padre del que está terminando recoja el código de terminación del mismo. Hasta que esto ocurra, estas estructuras se mantendrán y el proceso seguirá existiendo en estado terminado.



Estado de los procesos

Transición: Cuando la operación que mantiene a un proceso en estado bloqueado termina, el proceso puede haber perdido parte de los recursos que necesita para proseguir su ejecución. Este es por ejemplo el caso de un sistema con memoria virtual, en el que parte de las páginas de memoria del proceso han sido descargadas a disco. En dicho caso, el proceso tendría que pasar por un estado intermedio transición mientras recupera dichos recursos, y una vez que todos sus recursos vuelven a estar disponibles, volvería al estado preparado.



Ciclo de Vida de un proceso

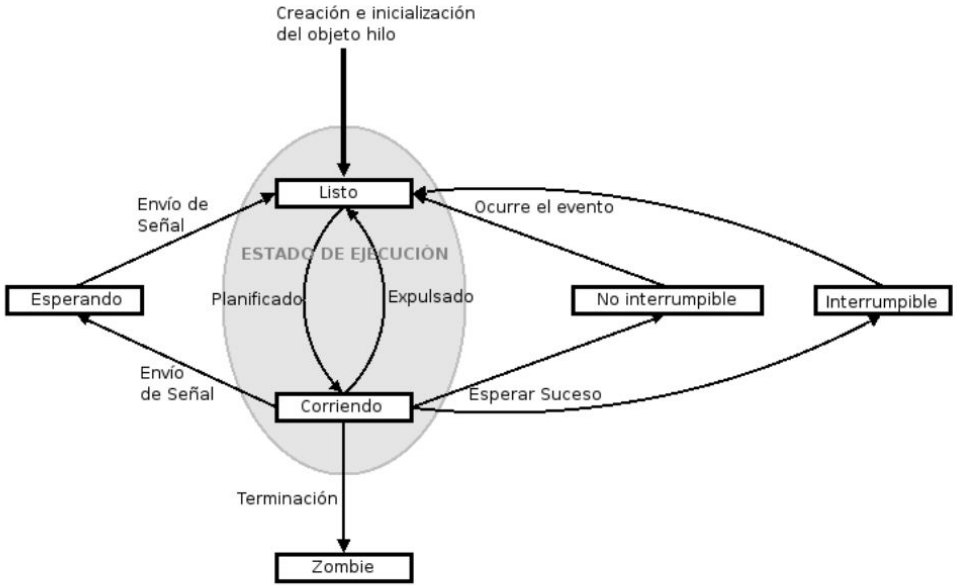
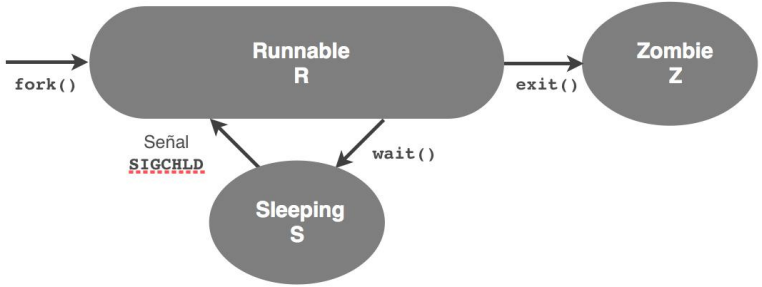


Figura 7.2: Estados de un procesos en Linux



Planificación de procesos

La planificación de procesos se refiere a cómo determina el sistema operativo al orden en que irá cediendo el uso del procesador a los procesos que lo vayan solicitando, y a las políticas que emplea para que el uso que den a dicho tiempo no sea excesivo respecto al uso esperado del sistema.



Planificación de procesos

A largo plazo

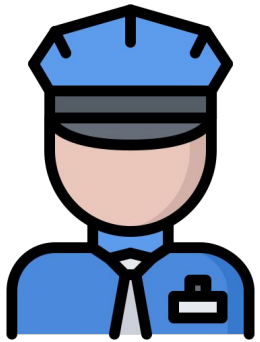
Decide qué procesos serán los siguientes en ser iniciados. Este tipo de planificación era el más frecuente en los sistemas de lotes y multiprogramados en lotes; las decisiones eran tomadas principalmente considerando los requisitos pre-declarados de los procesos y los que el sistema tenía libres al terminar algún otro proceso. La planificación a largo plazo puede llevarse a cabo con periodicidad de una vez cada varios segundos, minutos e inclusive horas.



Planificación de procesos

A mediano plazo

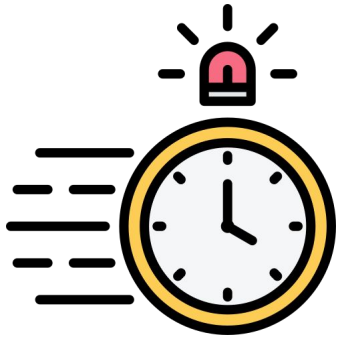
Decide cuáles procesos es conveniente bloquear en determinado momento, sea por escasez/saturación de algún recurso (como la memoria caché de nivel 1) o porque están realizando alguna solicitud que no puede satisfacerse momentáneamente; se encarga de tomar decisiones respecto a los procesos conforme entran y salen del estado de bloqueado.



Planificación de procesos

A corto plazo

Decide cómo compartir momento a momento al equipo entre todos los procesos que requieren de sus recursos, especialmente el procesador. La planificación a corto plazo se lleva a cabo decenas de veces por segundo (razón por la cual debe ser código muy simple, eficiente y rápido); es el encargado de planificar los procesos que están listos para ejecución.



Planificación de procesos

Proceso / Servicio	Descripción	Tipo de planificación	Justificación
systemd (Linux) / **services.exe** (Windows)	Proceso inicial que lanza el resto de servicios del sistema.	?	?
cron / **Task Scheduler**	Programa que planifica tareas periódicas (por ejemplo, copias de seguridad).	?	?
firefox / **chrome.exe**	Navegador abierto por el usuario.	?	?

Planificación de procesos

Proceso / Servicio	Descripción	Tipo de planificación	Justificación
systemd (Linux) / **services.exe** (Windows)	Proceso inicial que lanza el resto de servicios del sistema.	**Largo plazo**	Se inicia al arrancar el sistema y gestiona el resto de procesos.
cron / **Task Scheduler**	Programa que planifica tareas periódicas (por ejemplo, copias de seguridad).	**Largo plazo**	Se ejecuta constantemente y lanza trabajos nuevos según una planificación.
firefox / **chrome.exe**	Navegador abierto por el usuario.	**Corto plazo**	Compite por la CPU y es planificado continuamente por el kernel.

Planificación de procesos

Proceso / Servicio	Descripción	Tipo de planificación	Justificación
sshd / **Remote Desktop Service**	Servicio que espera conexiones remotas.	?	?
swapper/0 (Linux) / **Memory Compression** (Windows)	Gestiona procesos suspendidos y memoria virtual.	?	?
update-manager / **Windows Update Service**	Busca y aplica actualizaciones del sistema.	?	?
bash / **cmd.exe** / **powershell.exe**	Intérprete de comandos abierto por el usuario.	?	?

Planificación de procesos

Proceso / Servicio	Descripción	Tipo de planificación	Justificación
sshd / **Remote Desktop Service**	Servicio que espera conexiones remotas.	**Largo plazo**	Es un proceso residente, gestionado desde el inicio hasta el apagado.
swapper/0 (Linux) / **Memory Compression** (Windows)	Gestiona procesos suspendidos y memoria virtual.	**Medio plazo**	Interviene cuando hay que suspender o reanudar procesos desde disco.
update-manager / **Windows Update Service**	Busca y aplica actualizaciones del sistema.	**Largo plazo**	Se ejecuta en segundo plano o programado por el sistema.
bash / **cmd.exe** / **powershell.exe**	Intérprete de comandos abierto por el usuario.	**Corto plazo**	Usa CPU mientras el usuario ejecuta tareas interactivas.

Cambios de contexto

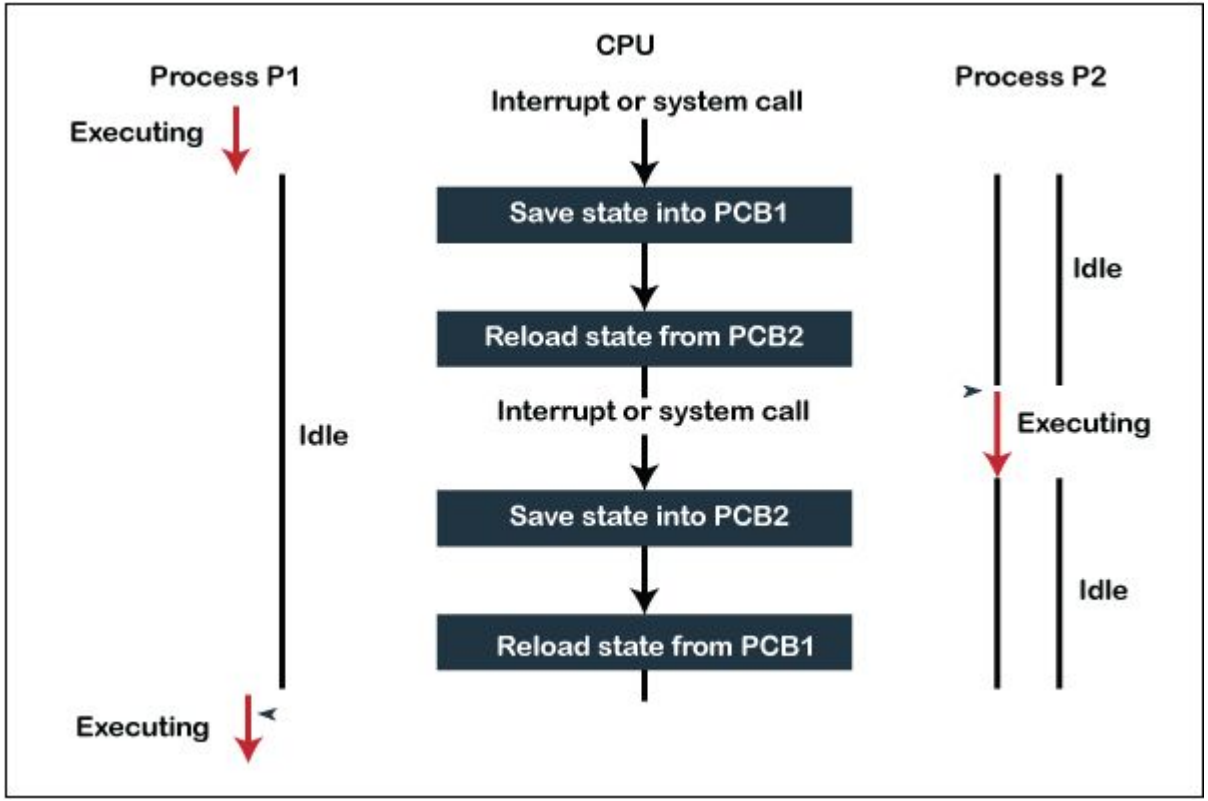
Un cambio de contexto consiste en la ejecución de una rutina perteneciente al núcleo del sistema operativo multitarea de una computadora, cuyo propósito es parar la ejecución de un hilo o proceso para dar paso a la ejecución de otro distinto.



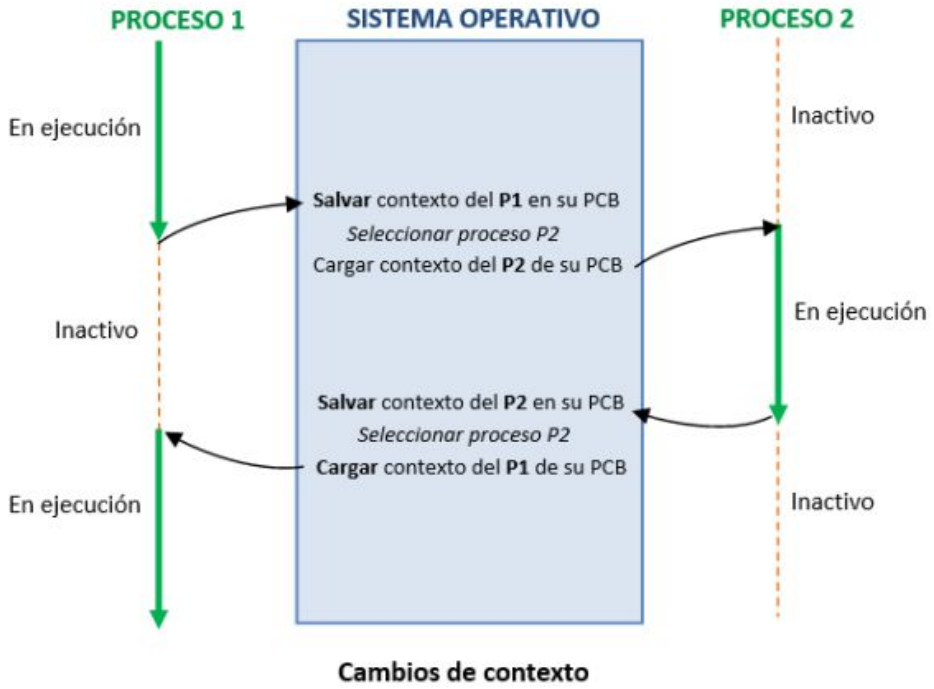
Pasos necesarios

1. Salvar el estado del programa que se estaba ejecutando. El estado, también denominado contexto, consiste en los valores de todos los registros del microprocesador. Se copian en la memoria principal.
2. Seleccionar otro programa para ejecutar. Entre todos los programas que estén preparados para ejecutarse, la rutina selecciona uno de ellos siguiendo algún algoritmo equitativo.
3. Restaurar el estado del programa seleccionado. Para ello, se toma el estado previamente copiado en la memoria principal y se vuelca en los registros del microprocesador.
4. Ejecutar el programa seleccionado. La rutina termina su ejecución saltando a la instrucción que estaba pendiente de ejecutar en el programa seleccionado.

Cambios de contexto



Cambios de contexto



Cambios de contexto

Pasos necesarios

1. **Salvar el estado del programa que se estaba ejecutando.** El estado, también denominado contexto, consiste en los valores de todos los registros del microprocesador. Se copian en la memoria principal.
2. **Seleccionar otro programa para ejecutar.** Entre todos los programas que estén preparados para ejecutarse, la rutina selecciona uno de ellos siguiendo algún algoritmo equitativo.
3. **Restaurar el estado del programa seleccionado.** Para ello, se toma el estado previamente copiado en la memoria principal y se vuelca en los registros del microprocesador.
4. **Ejecutar el programa seleccionado.** La rutina termina su ejecución saltando a la instrucción que estaba pendiente de ejecutar en el programa seleccionado.

Y cómo funciona esto

fork

El sistema operativo utiliza una identificación única para cada proceso para realizar un seguimiento de todos los procesos. Y para eso, fork() no toma ningún parámetro y devuelve un valor int de la siguiente manera:

Valor positivo

Si todo va bien, retorna al padre el PID del hijo

Zero (0)

Si todo va bien returna 0 al hijo recien creado

Valor negativo

Si la creación de un proceso hijo no tuvo éxito.

A probar

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{

    // make two process which run same
    // program after this instruction
    pid_t p = fork();
    if(p<0){
      perror("fork fail");
      exit(1);
    }
    printf("Hello world!, process_id(pid) = %d \n",getpid());
    return 0;
}
```

Para compilar: gcc -o nombreDelPrograma Programa.c

A probar

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    fork();
    fork();
    fork();
    printf("hello\n");
    return 0;
}
```

```
fork ();   // Line 1
fork ();   // Line 2
fork ();   // Line 3
       L1       // There will be 1 child process
    /     \     // created by line 1.
  L2      L2    // There will be 2 child processes
 /  \    /  \   //  created by line 2
L3  L3  L3  L3  // There will be 4 child processes
                // created by line 3
```

Para compilar: gcc -o nombreDelPrograma Programa.c

A probar


```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
    int x = 1;
    pid_t p = fork();
      if(p<0){
      perror("fork fail");
      exit(1);
    }
    else if (p == 0)
        printf("Child has x = %d\n", ++x);
    else
        printf("Parent has x = %d\n", --x);
}
```

```
int main()
{
    forkexample();
    return 0;
}
```

Para compilar: gcc -o nombreDelPrograma Programa.c

A probar

GATE | CS | 2008 | Operating System | System Calls | Question 66

Last Updated : Jun 21, 2025

Discuss

A process executes the following code

```
for (i = 0; i < n; i++) fork();
```

The total number of child processes created is

(A) n

(B) $2^n - 1$

(C) 2^n

(D) $2^{(n+1)} - 1$

This question is part of this quiz :
OS Process Management, GATE CS 2008, Operating system GATE CS PYQ QUIZ, System Calls GATE CS PYQ Quiz

https://www.geeksforgeeks.org/questions/a-process-executes-the-following-code-for-i/

Crear Zombies

```
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    // Fork returns process id
    // in parent process
    pid_t child_pid = fork();
    // Parent process
    if (child_pid > 0)
        sleep(50);
    // Child process
    else
        exit(0);
    return 0;
}
```



https://www.geeksforgeeks.org/c/zombie-and-orphan-processes-in-c/

Crear Huérfanos

```
#include<stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    // Create a child process
    int pid = fork();
    if (pid > 0)
        printf("in parent process");
    else if (pid == 0)
    {
        sleep(30);
        printf("in child process");
    }

    return 0;
}
```

https://www.geeksforgeeks.org/c/zombie-and-orphan-processes-in-c/



Algoritmos de planificación

El planificador a corto plazo puede ser invocado cuando un proceso se encuentra en algunas de las cuatro siguientes circunstancias:

- Pasa de estar ejecutando a estar en espera (por ejemplo, por solicitar una operación de E/S, esperar a la sincronización con otro proceso, etc.)
- Pasa de estar ejecutando a estar listo (por ejemplo, al ocurrir una interrupción
- Deja de estar en espera a estar listo (por ejemplo, al finalizar la operación de E/S que solicitó)

Finaliza su ejecución, y pasa de ejecutando a terminado

Algoritmos de planificación

En el primer y cuarto casos, el sistem operativo siempre tomará el control; un sistema que opera bajo multitarea preventiva implementará también el segundo y tercer casos, mientras que uno que opera bajo multitarea cooperativa no necesariamente reconocerá dichos estados.

Un proceso siempre abandonará la cola de procesos listos al requerir de un servicio del sistema.

Para todos los ejemplos a continuación, asumamos que los tiempos están dados en ticks; no nos preocupa en este caso a cuánto tiempo de reloj estos equivalen, sino el rendimiento relativo del sistema entero ante una carga dada.

Algoritmos de planificación



Triaje de Procesos

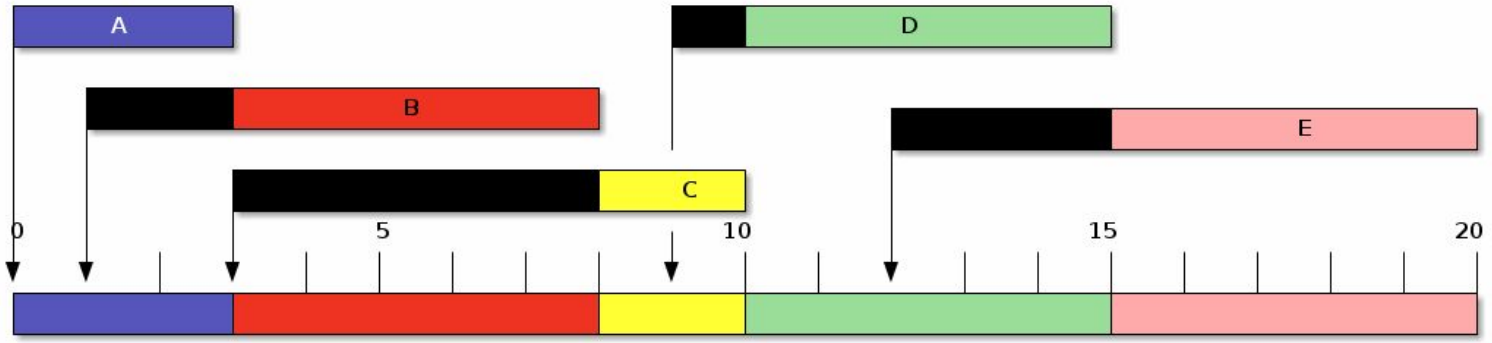
Algoritmos de planificación

Proceso	Tiempo de Llegada	*t*	Inicio	Fin	*T*	*E*	*P*
A	0	3	0	3	3	0	1
B	1	5	3	8	7	2	1.4
C	3	2	8	10	7	5	3.5
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio		4			6.2	2.2	1.74

https://sistop.gwolf.org/html/03_planificacion_de_procesos.html#sec-2

T	**Tiempo de retorno (Turnaround time)**	Es el **tiempo total que tarda un proceso desde que llega hasta que termina**. Fórmula: `T = Tiempo de finalización - Tiempo de llegada`
E	**Tiempo de espera (Waiting time)**	Es el **tiempo que el proceso pasa esperando en la cola de listos antes de ser ejecutado**. Fórmula: `E = T - Tiempo de ejecución (t)`
P	**Índice de penalización (Penalty ratio)** o **Índice de rendimiento**	Mide qué tan eficiente fue la ejecución del proceso en relación con su tiempo de servicio. Fórmula: `P = T / t`

Primero llegado, primero servido (FCFS)



Round Robin

El esquema ronda busca dar una relación de respuesta buena tanto para procesos largos como para los cortos. La principal diferencia entre la ronda y FCFS es que en este caso sí emplearemos multitarea preventiva:

A cada proceso que esté en la lista de procesos listos lo atenderemos por un sólo una unidad

Si un proceso no ha terminado de ejecutar al final de su quantum, será interrumpido y puesto al final de la lista de procesos listos, para que espere a su turno nuevamente. Los procesos que nos entreguen los planificadores a mediano o largo plazo se agregaron también al final de esta lista.

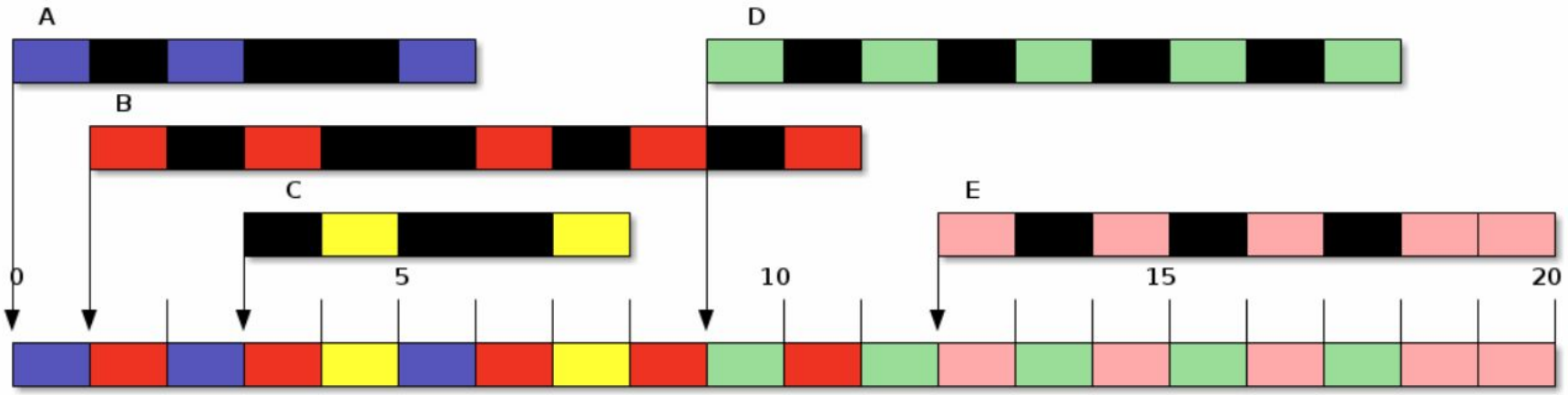
https://sistop.gwolf.org/html/03_planificacion_de_procesos.html#sec-2

Round Robin

Proceso	Tiempo de	*t*	Inicio	Fin	*T*	*E*	*P*
	Llegada						
A	0	3	0	6	6	3	2.0
B	1	5	1	11	10	5	2.0
C	3	2	4	8	5	3	2.5
D	9	5	9	18	9	4	1.8
E	12	5	12	20	8	3	1.6
Promedio		4			7.6	3.6	1.98

https://sistop.gwolf.org/html/03_planificacion_de_procesos.html#sec-2

Round Robin

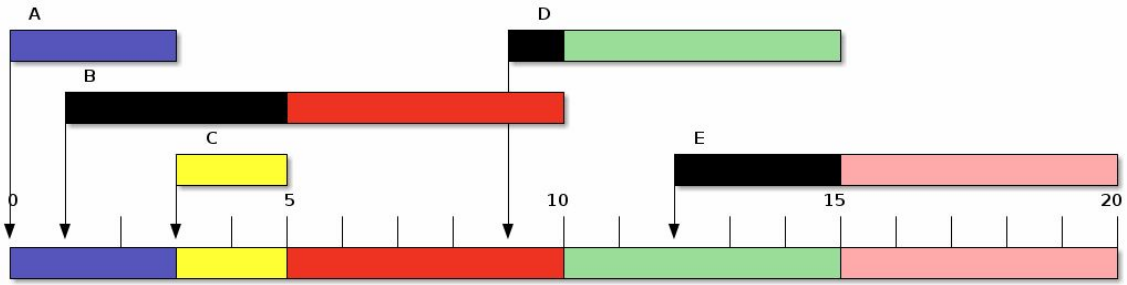


Ronda (*Round Robin*)

El proceso más corto a continuación (SPN)

Proceso	Tiempo de Llegada	t	Inicio	Fin	T	E	P
A	0	3	0	3	3	0	1.0
B	1	5	5	10	9	4	1.8
C	3	2	3	5	2	0	1.0
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio	—	4	—	—	**5.6**	**1.6**	**1.32**

El proceso más corto a continuación (SPN)



El proceso más corto a continuación (SPN)

Ronda Egoísta

Proceso	Tiempo de Llegada	*t*	Inicio	Fin	*T*	*E*	*P*
A	0	3	0	4	4	1	1.3
B	1	5	2	10	9	4	1.8
C	3	2	6	9	6	4	3.0
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio		4			6.6	2.6	1.79

Ronda egoísta (SRR)

El algoritmo **Round Robin (RR)** clásico asigna a cada proceso un **quantum de CPU fijo** (por ejemplo, 2 ms). Cada proceso se ejecuta ese tiempo y, si no termina, vuelve al final de la cola.

El algoritmo **Ronda Egoísta (Selfish RR)** cambia esa regla:

- Los procesos **ganan prioridad cuanto más tiempo llevan esperando**.
- Los procesos **nuevos tienen menos prioridad**.
- De esta forma, **los procesos que llevan más tiempo en el sistema se vuelven "egoístas"** y terminan antes de seguir esperando.

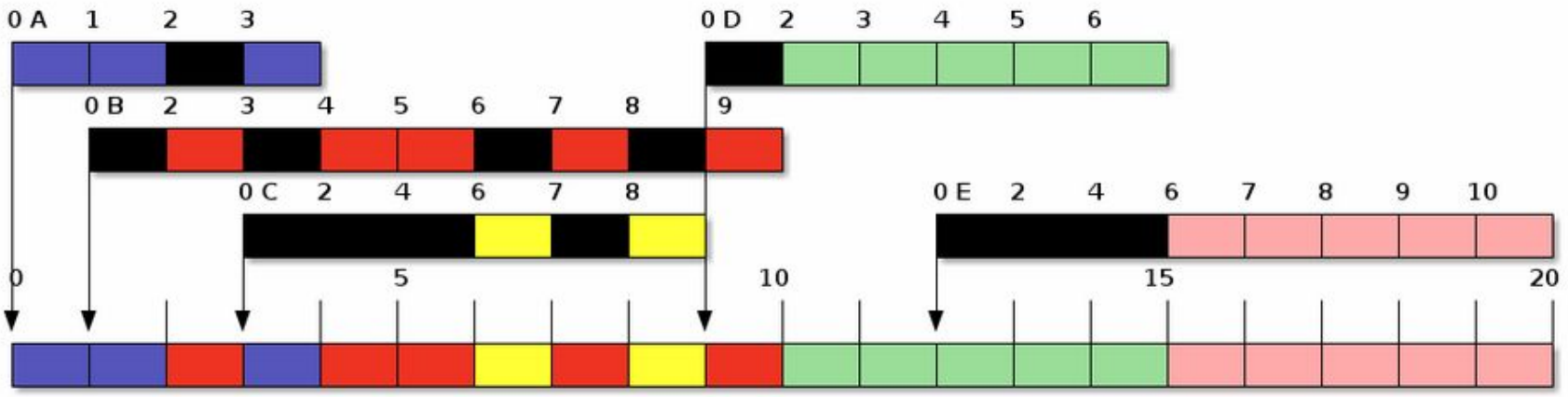
Ronda egoísta (SRR)

$$P_i(t) = 1 + \frac{T_{espera}(i)}{T_{servicio}(i)}$$

Donde:

- $P_i(t)$: prioridad dinámica del proceso i en el instante t.
- $T_{espera}(i)$: tiempo que el proceso ha estado esperando.
- $T_{servicio}(i)$: tiempo de CPU que necesita (o ya ha usado).

Ronda egoísta (SRR)



Ver los procesos en ejecución

Existen varias herramientas para ver los procesos en ejecución. La más importante es el comando ps.

ps (process status): lista los procesos con su PID, datos de usuario, identificador del proceso y línea de comandos usada.

```
$ ps
  PID TTY          TIME CMD
 6368 pts/0    00:00:00 bash
 7441 pts/0    00:00:00 ps
```

sin opciones, ps sólo muestra los procesos lanzados desde el terminal actual y con el mismo EUID que el usuario que lo lanzó.

Señales en linux

Signal	Value	Action	Comment
SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from abort(3)
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal

Señales en linux

Signal	Value	Action	Comment
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from alarm(2)
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at terminal

Opciones de ps:

1. opciones UNIX: pueden agruparse y se preceden por un guión: ps -ef
2. opciones BSD: pueden agruparse y van sin guión: ps uxa
3. opciones largas GNU: precedidas de dos guiones: ps - -user tomas

Algunas opciones:

- -e o ax muestra todos los procesos
- -u (o U o - -user) usuario: muestra los procesos de un usuario
- u: salida en formato usuario
- j: salida en formato job (muestra PID, PPID, etc.)
- -f o l: salida en formato largo
- f: muestra un árbol con la jerarquía de procesos
- k (o - - sort) campo: ordena la salida por algún campo (p.e. ps uxak rss)
- -o (o o o –format) formato: permite definir el formato de salida ps -o ruser, pid, comm=Comando

Opciones de ps:

1. opciones UNIX: pueden agruparse y se preceden por un guión: ps -ef
2. opciones BSD: pueden agruparse y van sin guión: ps uxa
3. opciones largas GNU: precedidas de dos guiones: ps - -user tomas

Algunas opciones:

- -e o ax muestra todos los procesos
- -u (o U o - -user) usuario: muestra los procesos de un usuario
- u: salida en formato usuario
- j: salida en formato job (muestra PID, PPID, etc.)
- -f o l: salida en formato largo
- f: muestra un árbol con la jerarquía de procesos
- k (o - - sort) campo: ordena la salida por algún campo (p.e. ps uxak rss)
- -o (o o o –format) formato: permite definir el formato de salida ps -o ruser, pid, comm=Comando

Ejemplo:

```
$ ps axu
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.0 1516 536 ? S 09:43 0:00 init [2]
root 2 0.0 0.0 0 0 ? S 09:43 0:00 [migration/0]
root 3 0.0 0.0 0 0 ? SN 09:43 0:00 [ksoftirqd/0]
root 4 0.0 0.0 0 0 ? S 09:43 0:00 [migration/1]
……
tomas 6475 0.1 4.9 140180 50920 ? S1 09:51 0:18 /usr/lib/mozilla-thunderbird
tomas 6528 0.2 3.6 116396 37948 ? Sl 10:01 0:25 /usr/lib/mozilla-firefox/firefox
```

en este ejemplo:

- %CPU y %MEM: porcentajes de uso de CPU y memoria
- VSZ: memoria virtual del proceso, en KBytes

- RSS: tamaño de la memoria residente (resident set size) es BKytes
- STAT: estado del proceso; puede ser:

Código	Significado
D	Uninterruptible sleep (usualmente IO)
R	Ejecutándose (running) o cola de ejecución
S	Interruptible sleep (p.e. esperando un evento)
T	Detenido
Z	Proceso zombie

Cuando se usa formato BSD puede aparecer otro código acompañando al principal:

Código	Significado
<	alta prioridad
N	baja prioridad
L	páginas bloqueadas (locked) en memoria slíder de sesión
l	multi-threaded
+	proceso en foreground

pstree muestra el árbol de procesos (similar a ps f)

```
init-+-acpid
     |-atd
     |-bonobo-activat
     |-clock-applet
     |-cron
     |-cupsd
     |-dbus-daemon-1
     |-dcopserver
     |-dirmngr
     |-2*[esd]
     |-events/0-       +-aio/0
     |                 |-ata/0
     |                 |-ata/1
     |                 |-kblockd/0
     |                 |-khelper
     |                 `-pdflush
     |-events/1-       +-aio/1
     |                 |-kacpid
     |                 |-kblockd/1
     |                 `-pdflush
     |-exim4
     |-famd
     |-firefox-bin---wvMime---gg
     v
```

top: ps da una versión estática de los procesos, top nos da una lista actualizada a intervalos

top - 17:34:08 up 7:50, 6 users, load average: 0.12, 0.31, 0.27
Tasks: 111 total, 1 running, 110 sleeping, 0 stopped, 0 zombie
Cpu(s): 6.2% us, 2.0% sy, 0.0% ni, 91.0% id, 0.0% wa, 0.8% hi, 0.0% si
Mem: 1026564k total, 656504k used, 370060k free, 65748k buffers
Swap: 2048248k total, 0k used, 2048248k free, 336608k cached

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
6130	root	15	0	63692	48m	9704	S	8.7	4.9	8:03.34	XFree86
6341	tomas	15	0	14692	8852	6968	S	4.3	0.9	1:55.13	metacity
6349	tomas	16	0	32792	14m	9232	S	1.3	1.5	0:41.60	gnome-terminal
6019	tomas	15	0	7084	3184	1896	D	0.3	0.3	0:23.22	famd
6401	tomas	15	0	16756	8280	6856	S	0.3	0.8	0:02.49	geyes_applet2
6427	tomas	15	0	18288	10m	8112	S	0.3	1.0	0:09.04	wnck-applet
7115	tomas	15	0	26312	13m	11m	S	0.3	1.4	0:00.61	kio_uiserver
7390	tomas	15	0	45016	30m	18m	S	0.3	3.0	0:38.69	kile
1	root	16	0	1516	536	472	S	0.0	0.1	0:00.61	init

- En la cabecera nos muestra un resumen del estado del sistema.
    - hora actual, tiempo que el sistema lleva encendido, el número de usuarios conectados y la carga media del sistema para los últimos 1, 5 y 15 minutos
    - número total de tareas y resumen por estado
    - estado de ocupación de la CPU y la memoria
- Por defecto, los procesos se muestran ordenados por porcentaje de uso de CPU (los más costosos arriba)
- Pulsando h mientras se ejecuta top, obtenemos una lista de comandos interactivos

- Algunos campos de top
  - VIRT: tamaño total del proceso (código, datos y librerías compartidas cargadas), VIRT=SWAP+RES
  - SWAP: memorias que ha sido swapped out o que aún no ha sido cargada
  - RES: memoria residente (RAM ocupada por el proceso)
  - CODE y DATA: memoria ocupada por el código y datos (datos y pila, pero no librerías compartidas) del proceso
  - SHR: memoria compartida (memoria que puede ser compartida con otros procesos)
  - P: última CPU usada (SMP)
  - nFLT: número de fallos de página para el proceso

strace: Muestra las llamadas al sistema realizadas por un proceso en ejecución

- Ejemplo de un strace sobre un top en ejecución

$ strace top

```
gettimeofday({1195811866, 763977}, {4294967236, 0}) = 0
open("/proc/meminfo", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7f55000
read(3, "MemTotal: 2066348 kB\nMemFre"..., 1024) = 728
close(3) = 0
munmap(0xb7f55000, 4096) = 0
open("/proc", O_RDONLY|O_NONBLOCK|O_LARGEFILE|O_DIRECTORY) = 3
fstat64(3, {st_mode=S_IFDIR|0555, st_size=0, ...}) = 0
fcntl64(3, F_SETFD, FD_CLOEXEC) = 0
getdents(3, /* 52 entries */, 1024) = 1020
getdents(3, /* 64 entries */, 1024) = 1024
stat64("/proc/1", {st_mode=S_IFDIR|0555, st_size=0, ...}) = 0
open("/proc/1/stat", O_RDONLY) = 4
read(4, "1 (init) S 0 1 1 0 -1 4194560 44"..., 1023) = 185
close(4)
.....
```

Manipulación de procesos

Ya hemos visto que mediante Ctrl-C y Ctrl-Z podemos terminar y detener un proceso:

- esas combinaciones envían señales a los procesos
- el comando básico para enviar señales a un proceso es kill

El comando kill: Permite enviar señales a los procesos

- kill -l lista el conjunto de señales

```
$ kill -l
1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL
5) SIGTRAP      6) SIGABRT      7) SIGBUS       8) SIGFPE
9) SIGKILL      10) SIGUSR1     11) SIGSEGV     12) SIGUSR2
13) SIGPIPE     14) SIGALRM     15) SIGTERM     17) SIGCHLD
18) SIGCONT     19) SIGSTOP     20) SIGTSTP     21) SIGTTIN
22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO
30) SIGPWR      31) SIGSYS ....
```

Sintaxis de kill

kill [señal] PID

- señal puede indicarse mediante el n´umero o el c´odigo:
  kill -9 y kill -KILL son equivalentes
- las señales más comunes son:
  - SIGHUP (1): cuelgue del terminal o muerte del proceso controlador
  - SIGTERM (15): mata el proceso permitiéndole terminar correctamente
  - SIGKILL (9): mata el proceso sin permitirle terminar
  - SIGSTOP (19): para el proceso
  - SIGCONT (18): contin´ua si parado
  - SIGINT (2): interrupción de teclado (Ctrl-C)
  - SIGTSTP (20): stop de teclado (Ctrl-Z)
  - SIGQUIT (3): salida de teclado (Ctrl-\)

Algunas características de las señales:

- La señal que se envía por defecto es TERM (15)
  - Los procesos pueden ignorar esta señal y no terminar
  - Las señales KILL (9) y STOP (19) no pueden ignorarse
- En bash, cuando enviamos una señal SIGHUP a un shell, este se lo reenvía a todos sus hijos
- Cuando cerramos un terminal en un entorno gráfico, o abandonamos una sesión, se envía un SIGHUP a todos sus hijos
- La mayoría de los demonios (daemons) responden a la se˜nal SIGHUP volviendo a leer sus ficheros de configuraci´on:
  - en vez de matar y reiniciar un demonio podemos hacer un kill -HUP para reiniciarlo

Otros Comandos

pgrep: busca en la lista de procesos para localizar el PID a partir del nombre (similar a ps | grep)

- Ejemplo:

```
$ pgrep -u root sshd # PID del proceso sshd de root
```

pkill: permite enviar señales a los procesos indicándolos por nombre en vez de por PID

- Ejemplo:

```
$ pkill -9 proceso
```

- si hay varios procesos con el mismo nombre los mata a todos
- en vez de un nombre admite un patrón (p.e. pkill ´l.*´)
    - tener cuidado con su uso (es fácil matar procesos de forma errónea)

killall: similar a pkill, pero no admite patrones en el nombre, y tiene otras opciones

nohup: normalmente, cuando salimos de un login shell (logout) o cerramos una un terminal, se envía una señal SIGHUP a todos los procesos hijos:

- si lanzamos un proceso en background y salimos de la sesión el proceso se muere al morir el shell desde el que lo iniciamos

El comando nohup permite que un lanzar un comando ignorando las señales SIGHUP

nohup comando

- la salida del comando se redirige al fichero nohup.out

exec: como ya vimos, ejecuta un comando reemplazando al shell desde el que se lanza:

- es otra forma de crear un proceso que no se muere al morirse su shell.

Ejemplo:

```
$ yes >/dev/null &
[1] 9848
$ yes >/dev/null &
[2] 9849
$ ps
PID TTY TIME CMD
9834 pts/7 00:00:00 bash
9848 pts/7 00:00:02 yes
9849 pts/7 00:00:01 yes
9850 pts/7 00:00:00 ps
$ kill -STOP 9849   yes >/dev/null
$ jobs
[1]- Running        yes >/dev/null &
[2]+ Running        yes >/dev/null &
$ kill -KILL 9848
$ kill -1 9849
[1]- Matado         yes >/dev/null
[2]+ Colgar         yes >/dev/null
```

Ejemplo:

```
$ yes > /dev/null &
[1] 14724
$ yes > /dev/null &
[2] 14725
$ ps
  PID  TTY          TIME CMD
 7083 pts/3    00:00:00 bash
14724 pts/3    00:00:02 yes
14725 pts/3    00:00:02 yes
14726 pts/3    00:00:00 ps
$ pgrep yes
14724
14725
$ pkill -9 yes
$ ps
  PID  TTY          TIME CMD
 7083 pts/3    00:00:00 bash
14730 pts/3    00:00:00 ps
[1]- Matado                yes > /dev/null
[2]+ Matado                yes > /dev/null
```

Ejemplo:

```
$ nohup yes > /dev/null &
[1] 9620
$ kill -HUP 9620
$ ps
  PID  TTY          TIME CMD
 8293 pts/5    00:00:00 bash
 9620 pts/5    00:00:13 yes
 9621 pts/5    00:00:00 ps
$ kill 9620
[1]+                       nohup yes > /dev/null
Terminado
```

Manejo de la prioridad y recursos de un proceso

Cuando un proceso se ejecuta, lo hace con una cierta prioridad

- las prioridades van desde -20 (prioridad más alta) a 19 (prioridad más baja)
- por defecto, los procesos se ejecutan con prioridad 0
  - un usuario normal solo puede asignar prioridades más bajas (números positivos)
  - root puede asignar prioridades más altas (números negativos)
- los comandos para manejo de prioridades son **nice y renice**

nice: permite lanzar un comando con una cierta prioridad

- Sintaxis

  nice -n ajuste comando

la prioridad por defecto se modifica por ajuste

- Ejemplo:

$ nice -n 10 abiword & # disminuye la prioridad en 10

$ ps -o pid,pri,ni,stat,cmd

PID	PRI	NI	STAT	CMD
7133	24	0	Ss	bash
7431	14	10	SN	abiword
7552	23	0	R+	ps -o pid,pri,ni,stat,cmd

$ nice -n -1 abiword

nice: no se puede establecer la prioridad: Permiso denegado

renice: permite cambiar la prioridad de un proceso que está en ejecución

Sintaxis:

renice pri [-p pid] [-g pgrp] [-u user]

- las opciones son:
  - -p pid cambia la prioridad para el proceso especificado
  - -g pgrp cambia la prioridad para los procesos ejecutados por los usuarios que pertenecen al grupo con ID=pgrp
  - -u user cambia la prioridad para los procesos del usuario especificado

Ejemplo:

$ abiword &
[1] 7681
$ renice 10 -p 7681
7681: old priority 0, new priority 10
$ renice 3 -u tomas
503: old priority 0, new priority 3

Control de los recursos de un proceso

El comando interno de bash ulimit permite controlar los recursos de los que dispone un proceso arrancado por el shell

Sintaxis

ulimit [opciones] [limite]

Algunas opciones:

- -a muestra los límites actuales
- ⋆ -f máximo tamaño de los ficheros creados por el shell (opción por defecto)
- ⋆ -n máximo número de ficheros abiertos
- ⋆ -s máximo tamaño de la pila
- ⋆ -t máximo tiempo de cpu
- ⋆ -S/-H usa los límites soft y hard
  - el usuario puede incrementar su límite blando, pero sin superar el límite duro
  - estos límites pueden ser fijados en el /etc/profile, /etc/bash.bashrc

Análisis básico del rendimiento del sistema

Además de ps y top existen comandos básicos que nos pueden mostrar el estado del sistema en cuanto a uso de CPU y consumo de memoria

uptime

Muestra la hora actual, el tiempo que el sistema lleva encendido,el número de usuarios conectados y la carga media del sistema para los últimos 1, 5, y 15 minutos (lo mismo que en la primera línea de top)

Ejemplo:

$ uptime
20:25:03 up 25 days, 11:12, 13 users, load average: 3.00, 3.07, 3.08

Además de la información dada por uptime, el comando **w** muestra información sobre los usuarios y sus procesos

Ejemplo:

$ w
20:24:52 up 25 days, 11:11, 13 users, load average: 3.10, 3.09, 3.08
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
paula pts/1 godello.dec.usc. 12:08pm 8:15m 8.39s 8.36s ssh -p 1301 -X paula@porto
paula pts/2 godello.dec.usc. 12:09pm 7:30 0.11s 0.11s bash
pichel pts/4 - 11:08am 7.00s 0.33s 0.33s -bin/tcsh
pichel pts/5 - 7:12pm 56:56 0.26s 0.16s ssh usceljpc@sc.cesga.es -X
pichel pts/6 - 4:35pm 21:15 16.61s 0.02s /bin/sh reordena.sh man68097.ccs tomas pts/8 jumilla.dec.usc. 8:24pm 0.00s 0.05s 0.02s w

Definiciones:

- LOGIN@ la hora a la que se conectó el usuario
- IDLE tiempo que lleva ocioso el terminal
- JCPU el tiempo de CPU consumido por los procesos que se ejecutan en el TTY
- PCPU tiempo consumido por el proceso actual (el que aparece en la columna WHAT)

free

Muestra la cantidad de memoria libre y usada en el sistema, tanto para la memoria física como para el swap, sí como los buffers usados por el kernel (similar a lo mostrado en la cabecera de top)

Ejemplo:

```
$ free          total  used        free          shared      buffers      cached 3098556
Mem:          2969388  129168      0             419300      1674492
-/+ buffers/cache:     875596                   2222960
Swap:         6144736  3129376     3015360
```

- la columna shared no significa nada (obsoleta)

Opciones:

- -b,-k,-m,-g memoria en bytes/KBytes/MBytes/GBytes
- -t muestra una l´ınea con el total de memoria (f´ısica + swap)
- -s delay muestra la memoria de forma continua, cada delay segundos

Herramientas gráficas

Además de los comandos comentados, si disponemos de un entorno X Windows podemos usar alguna herramienta gráfica en vez de top y ps:

- proporcionan una visión más clara y son fáciles de usar

  Algunas herramientas son:

  - **gnome-system-monitor:** visor de procesos y monitorizador de recursos del sistema de GNOME
  - **KDE System Guard (ksysguard):** gestor de tareas y monitor de rendimiento de KDE

El directorio /proc

Pseudo Filesystem que guarda información sobre el sistema y los procesos

- se inicializa durante el arranque
- está implementado en memoria y no se guarda en disco
- la estructura del directorio /proc depende de la versión del kernel
- los comandos vistos (ps, top, etc.) obtienen la información sobre los procesos de este directorio

Vimos algo del contenido de /proc (ficheros interrupts, ioports y dma); otros ficheros son:

- **cpuinfo:** información estática de la CPU
- **meminfo:** información de uso de la memoria
- **partitions:** información sobre las particiones
- **filesystems:** sistemas de ficheros soportados por el kernel
- **versión:** versión y fecha del kernel
- **bus/:** directorio con información de los buses PCI y USB
- **cmdline:** línea de arranque del kernel
- **devices:** dispositivos del sistema de caracteres o bloques
- **ide/:** directorio con información del bus IDE
- **modules:** módulos del kernel
- **net/:** directorio con información de red

Además, existen un directorio por cada proceso, que se identifica con el PID del proceso, en el que se puede encontrar información sobre cada proceso, incluidos:

- El directorio desde que se invocó el proceso (enlace cwd)
- Nombre del ejecutable (enlace exe) y la l´ınea de comandos con la que fue invocado (fichero cmdline)
- Entorno en que se ejecuta el proceso (fichero environ)
- Estado del proceso (fichero status)
- Descriptores de ficheros abiertos y archivos o procesos relacionados (directorio fd)
- Mapa de memoria (fichero maps)

Nota: en el kernel 2.6 de Linux aparece un nuevo pseudofilesystem (/sys) que reemplaza al /proc en lo que se refiere a visualización y ajuste de dispositivos

Gracias

